Srishyla Educational Trust (R), Bheemasamudra.

GM INSTITUTE OF TECHNOLOGY

(Affiliated to VTU Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)
Post Box No: 4 P.B. Road, Davangere-577006, Karnataka

DEPARTMENT OF INFORMATION SCIENCE & ENGG

FULL STACK DEVELOPMENT

(Subject Code: BIS601)

Lab Manual

[As per CBCS scheme, Effective from the academic year 2026 -2027]

For VI Semester

VISION & MISSION OF THE ISE DEPARTMENT

VISION

To evolve as an excellent Technological Education centre of Information Science to create competitive professional engineers for the advancement of society.

MISSION

- To provide Application Specific Training and Practical Experience for developing quality engineers in information science and Engineering.
- To inculcate creative skills in students to become competent in the fields of information science and engineering.
- To inculcate professional, Social and ethical values in the students to perform better in diverse environment.

Prepared By,

Ms. Sanjana R
Dept. of ISE

Reviewed By,

Dr. Neelambike S
Professor & Head, Dept. of ISE

What is Full Stack Development?

Full Stack Development refers to the development of both front-end (client-side) and back-end (server-side) portions of web applications. They handle the complete development process, from designing user interfaces to managing databases and server-side functions, making them versatile in building end-to-end applications.

Technology Related to Full Stack Development:

Front-end Development:

It is the visible part of website or web application which is responsible for user experience.
The user directly interacts with the front-end portion of the web application or website.

Front-end Technologies:

The front end portion is built by using some languages which are discussed below:

- **HTML:** HTML stands for HyperText Markup Language and is used to design the front-end structure of web pages. Hypertext defines links between pages, while markup language uses tags to define and structure the content.
- **CSS:** CSS, or Cascading Style Sheets, is a simple language used to make web pages visually presentable. It allows you to apply styles to web pages, and importantly, it works independently of the HTML structure.
- **JavaScript:** JavaScript is a famous scripting language used to create the magic on the sites to make the site interactive for the user. It is used to enhancing the functionality of a website to running cool games and web-based software.

Front End Libraries and Frameworks:

- **AngularJS:** AngularJS is an open-source JavaScript front-end framework mainly used for building single-page applications (SPAs). It turns static HTML into dynamic HTML, extends HTML with directives, and supports data binding. It is continuously growing and can be freely used and modified by anyone.
- **React.js:** React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It is an open-source, component-based front-end library focused on the view layer and is maintained by Facebook.
- **Bootstrap:** Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first web sites.
- **jQuery:** jQuery is an open-source JavaScript library that simplifies interactions between HTML/CSS (the DOM) and JavaScript. It streamlines DOM traversal and manipulation, event handling, animations, Ajax requests, and cross-browser development.
- **SASS:** It is the most reliable, mature and robust CSS extension language. It is used to extend the functionality of an existing CSS of a site including everything from variables, inheritance, and nesting with ease.

Back-end Technologies:

It refers to server-side development focused on how a website functions. It manages the database through queries and APIs triggered by client-side commands. Such websites consist of the front end, back end, and database, with the back end built using various libraries, frameworks, and languages:

- **PHP:** PHP is a server-side scripting language designed specifically for web development. Since, PHP code executed on server side so it is called server-side scripting language.
- **C++:** It is a general purpose programming language and widely used now a days for competitive programming. It is also used as backend language.
- **Java:** Java is one of the most popular and widely used programming language and platform. It is highly scalable. Java components are easily available.
- **Python:** Python is a programming language that lets you work quickly and integrate systems more efficiently.
- **Node.js:** Node.js is an open source and cross-platform runtime environment for executing JavaScript code outside of a browser. You need to remember that NodeJS is not a framework and it's not a programming language. Most of the people are confused and understand it's a framework or a programming language. We often use Node.js for building back-end services like APIs like Web App or Mobile App. It's used in production by large companies such as Paypal, Uber, Netflix, Walmart and so on.

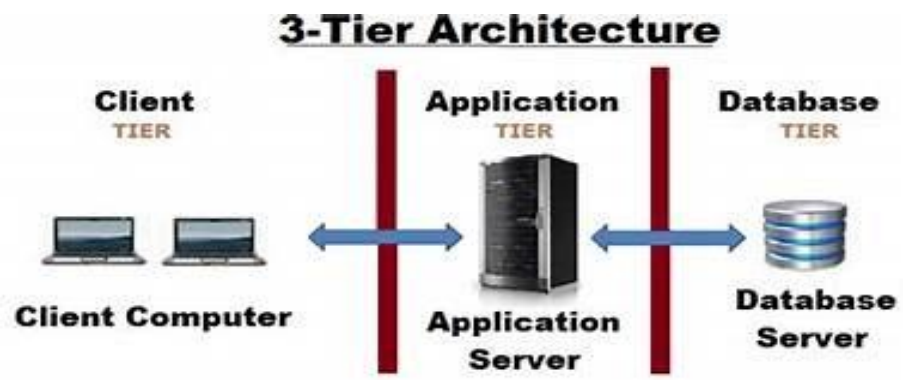
Back End Frameworks:

The list of backend frameworks are: Express, Django, Rails, Laravel, Spring etc.
The other backend program/scripting languages are: C#, Ruby, REST, GO etc.

Database:

Database is the collection of inter-related data which helps in efficient retrieval, insertion and deletion of data from database and organizes the data in the form of tables, views, schemas, reports etc.

- **Oracle:** Oracle database is the collection of data which is treated as a unit. The purpose of this database is to store and retrieve information related to the query. It is a database server and used to manages information.
- **MongoDB:** MongoDB, the most popular NoSQL database, is an open source document-oriented database. The term 'NoSQL' means 'non-relational'. It means that MongoDB isn't based on the table-like relational database structure but provides an altogether different mechanism for storage and retrieval of data.
- **SQL:** Structured Query Language is a standard Database language which is used to create, maintain and retrieve the relational database.



<u>3-Tier Architecture divides an application into three separate layers (tiers):</u>

1. Client Tier (Presentation Layer)
2. Application Tier (Business Logic Layer)
3. Database Tier (Data Layer)

Each tier has a specific responsibility, which makes systems scalable, secure, and easy to maintain.

Client Tier (Client Computer)

The **Client Tier**, also known as the **Presentation Layer**, is the first tier shown on the left side of the diagram. This tier represents the **client computer or user device** through which users interact with the application. It is responsible for displaying the user interface, such as web pages, forms, buttons, and input fields. The client tier collects user input and sends requests to the Application Tier. It does not contain business logic or direct access to the database; its main purpose is to provide a user-friendly interface and forward requests.

Application Tier (Application Server)

The **Application Tier**, is also called the **Business Logic Layer**. This tier acts as an intermediary between the Client Tier and the Database Tier. The application server processes all requests received from the client, applies business rules, validates data, and manages workflows. It determines how data should be processed and what operations should be performed on the database. By placing the logic in this tier, the system becomes more secure and easier to maintain, as clients cannot directly interact with the database.

Database Tier (Database Server)

The **Database Tier**, is the **Data Layer** of the architecture. This tier is responsible for storing, retrieving, and managing application data. It consists of a database server such as MySQL, Oracle, or PostgreSQL. The database tier executes queries sent by the Application Tier and returns the required results. It does not communicate directly with the client, which helps protect sensitive data and maintain data integrity.

PROGRAM - 1

1. a. Write a script that Logs "Hello, World!" to the console. Create a script that calculates the sum of two numbers and displays the result in an alert box.
b. Create an array of 5 cities and perform the following operations: Log the total number of cities. Add a new city at the end. Remove the first city. Find and log the index of a specific city.

Step 1: Install Visual Studio Code (VS Code)

Why Visual Studio Code? VS Code is a free, open-source code editor that provides powerful features like IntelliSense, debugging support, Git integration, and extensions specifically designed for JavaScript and Node.js development.

- Download VS Code
  - Go to the Visual Studio Code official website.
  - Download the version for your operating system.
- Install VS Code
  Follow the installation instructions for your operating system.
- Verify Installation
  After installation, open VS Code and check if it's working. If successful, you should see the VS Code welcome screen.

Step 2: Create Folder and open with vscode

- Inside the folder create index.html file.
- After then copy the below code and paste it in that index.html file and save it. Note do one by one both programs.

  PROGRAM:

```
<script>
console.log("Hello World!")
let number1 = 5;
let number2 = 7;
let sum = number1 + number2;
alert("The sum of the numbers is: " + sum);
</script>
```

Step 3: To run program:

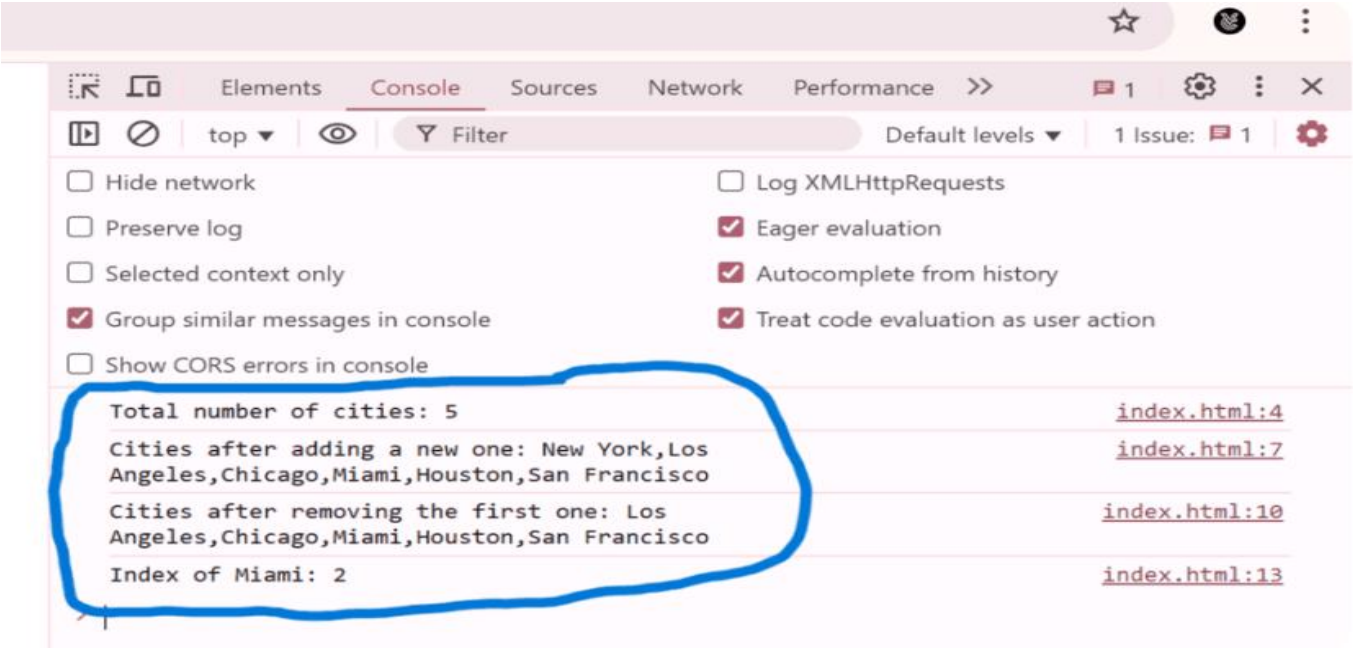
- Make sure Before running this program live server extension installed in your vscode to run.
- Right click in the code and open with live server to see the output in alert box.
- After then open the developer console and see the output of hello world!.

  OUTPUT:
  Hello, World!
  The sum is: 12

1b) Create an array of 5 cities and perform the following operations: Log the total number of cities. Add a new city at the end. Remove the first city. Find and log the index of a specific city.

```
<script>
let cities = ["New York", "Los Angeles", "Chicago", "Miami", "Houston"];

console.log("Total number of cities: " + cities.length);

cities.push("San Francisco");
console.log("Cities after adding a new one: " + cities);

cities.shift();
console.log("Cities after removing the first one: " + cities);

let cityIndex = cities.indexOf("Miami");
console.log("Index of Miami: " + cityIndex);
</script>
```

OUTPUT:



PROGRAM - 2

2. Read a string from the user, Find its length. Extract the word "JavaScript" using substring() or slice(). Replace one word with another word and log the new string. Write a function isPalindrome(str) that checks if a given string is a palindrome (reads the same backward).

```
// Read string from user
let str = prompt("Enter a sentence containing JavaScript");

// 1. Find length of string
console.log("Length of string:", str.length);

// 2. Extract the word "JavaScript"
let word = str.substring(str.indexOf("JavaScript"), str.indexOf("JavaScript") + 10);
console.log("Extracted word:", word);

// 3. Replace one word with another
let newStr = str.replace("JavaScript", "Python");
console.log("New String:", newStr);

// 4. Palindrome function
function isPalindrome(s) {
  let reversed = s.split("").reverse().join("");
  return s === reversed;
}

// Example check
let check = prompt("Enter a word to check palindrome");
if(isPalindrome(check)){
  console.log(check + " is a Palindrome");
}else{
  console.log(check + " is Not a Palindrome");
}
```

Output:

```
Enter a sentence containing JavaScript: I love JavaScript programming
Length of string:  29
Extracted word:  JavaScript
New String:  I love Python programming
Enter a word to check palindrome: madam
madam is a Palindrome

Enter a sentence containing JavaScript: I love JavaScript programming
Length of string:  29
Extracted word:  JavaScript
New String:  I love Python programming
Enter a word to check palindrome: java
java is Not a Palindrome
```

PROGRAM - 3

3. Create an object student with properties: name (string), grade (number), subjects (array), displayInfo() (method to log the student's details) Write a script to dynamically add a passed property to the student object, with a value of true or false based on their grade. Create a loop to log all keys and values of the student object.

```
let student = {
  name: "Rahul",
  grade: 75,
  subjects: ["Math", "Science", "English"],

  displayInfo: function () {
    console.log("Name:", this.name);
    console.log("Grade:", this.grade);
    console.log("Subjects:", this.subjects);
  }
};

student.displayInfo();

// Add passed property
student.passed = student.grade >= 50;

console.log("Passed:", student.passed);

// Print only properties (not functions)
for (let key in student) {
  if (typeof student[key] !== "function") {
    console.log(key + " : " + student[key]);
  }
}
```

Output:

```
Name: Rahul
Grade: 75
Subjects: [ 'Math', 'Science', 'English' ]
Passed: true

name : Rahul
grade : 75
subjects : Math,Science,English
passed : true
```

PROGRAM – 4

4. **Create a button in your HTML with the text "Click Me". Add an event listener to log "Button clicked!" to the console when the button is clicked. Select an image and add a mouseover event listener to change its border color. Add an event listener to the document that logs the key pressed by the user.**

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Event Listeners Example</title>
    <style>
        img {
            width: 200px;
            height: 140px;
            border: 5px solid black;
            margin-top: 10px;
        }

        button {
            padding: 5px 10px;
            background: #000;
            color: #fff;
            border-radius: 5px;
            cursor: pointer;
        }
    </style>
</head>

<body>
    <button id="myButton">Click Me</button><br>
    <img id="myImage" src="https://vtucircle.com/wp-content/uploads/2025/01/Full-Stack
Development-BIS601.jpg">

    <script>
        const button = document.getElementById('myButton');
        button.addEventListener('click', function () {
            console.log('Button clicked!');
        });
```

```
        const image = document.getElementById('myImage');
        image.addEventListener('mouseover', function () {
            image.style.borderColor = 'red';
        });

        document.addEventListener('keydown', function (event) {
            console.log('Key pressed: ' + event.key);
        });

    </script>
</body>
</html>
```

OUTPUT:

PROGRAM – 6

6. Create a component Counter with A state variable count initialized to 0. Create Buttons to increment and decrement the count. Simulate fetching initial data for the Counter component using useEffect (functional component) or componentDidMount (class component). Extend the Counter component to Double the count value when a button is clicked. Reset the count to 0 using another button.

Step 1: Create Project

- Create a folder and open with vscode.
- Open integrated vscode terminal.
- After then create a react app using below command.

npx create-react-app counter-app

After creating successfully then change the directory using below command.

cd counter-app

Step 2: Create File inside the src folder

Create two separate file Counter.js and CounterClass.js inside the src folder.
After then copy the below code and paste within the respective file Counter.js and CounterClass.js
After then Modify the App.js file present in the src folder. Copy the below code and paste it.
After then Modify the App.css file present in the src folder. Copy the below code and paste it.

Counter.js:

```
import React, { useState, useEffect } from 'react';
const Counter = () => {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('Fetching initial data...');
  }, []);

  const increment = () => setCount(prevCount => prevCount + 1);
  const decrement = () => setCount(prevCount => prevCount - 1);
  const doubleCount = () => setCount(prevCount => prevCount * 2);
  const resetCount = () => setCount(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
      <button onClick={doubleCount}>Double</button>
      <button onClick={resetCount}>Reset</button>
    </div>
  );
};

export default Counter;
```

CounterClass.js:

```
import React, { Component } from 'react';

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  componentDidMount() {
    console.log('Fetching initial data...');
  }
```

```
  increment = () => {
    this.setState(prevState => ({ count: prevState.count + 1 }));
  };
  decrement = () => {
    this.setState(prevState => ({ count: prevState.count - 1 }));
  };
  doubleCount = () => {
    this.setState(prevState => ({ count: prevState.count * 2 }));
  };
  resetCount = () => {
    this.setState({ count: 0 });
  };
  render() {
    return (
      <div>
        <h1>Count: {this.state.count}</h1>
        <button onClick={this.increment}>Increment</button>
        <button onClick={this.decrement}>Decrement</button>
        <button onClick={this.doubleCount}>Double</button>
        <button onClick={this.resetCount}>Reset</button>
      </div>
    );
  }
}
export default Counter;
```

App.js:

```
import React from 'react';
import './App.css';
import Counter from './CounterClass';

function App() {
  return (
    <div className="App">
      <h1 className='head'>Welcome to the Counter App</h1>
      <Counter />
    </div>
  );
}

export default App;
```

App.css:

```
.App {
  border-radius: 7px;
  background: #ffeaea;
  text-align: center;
}

#root {
  margin: 20px auto;
  width: 40%;
  height: 400px;
}

h1 {
  color: #fff;
  font-size: 25px;
  border-radius: 6px;
  margin: 20px auto;
  background: #000000;
  width: fit-content;
  padding: 10px 60px;
}

.head {
  width: 100%;
  font-size: 25px;
  margin: 0;
  border-radius: 6px;
  color: #fff;
  background: #000000;
  padding: 10px 0;
}

button {
  margin: 10px;
  padding: 8px 10px;
  font-size: 16px;
  border: none;
  font-weight: 600;
  border-radius: 5px;
  cursor: pointer;
  background: #e02020;
  color: #fff;
}
```

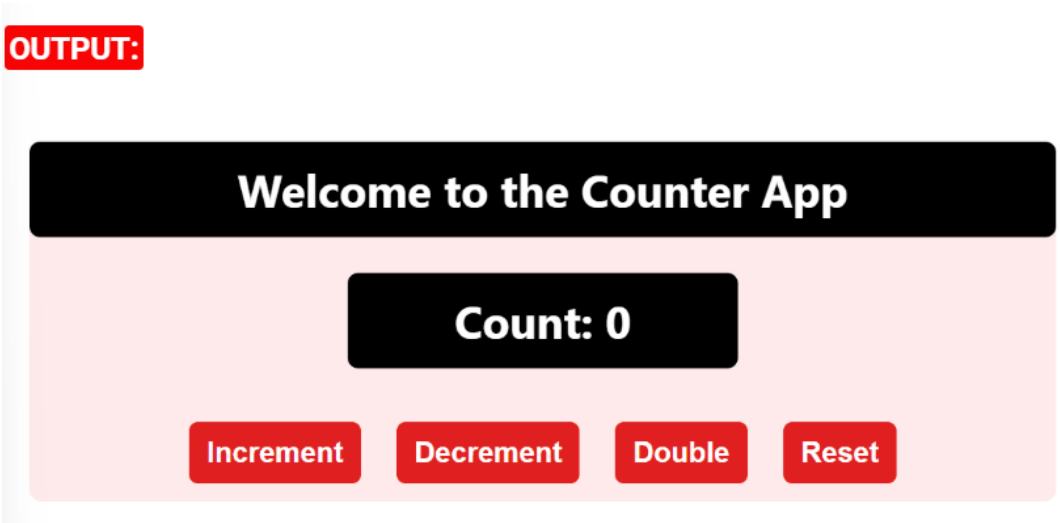
Step 3: Start the Development Server

Now, you can start the development server by using below command:

npm start

This will start a local development server and automatically open your new React application in your default web browser. The server will reload the page automatically whenever you make changes to your code.

OUTPUT:



PROGRAM – 5

Build a React application to track issues. Display a list of issues (use static data). Each issue should have a title, description, and status (e.g., Open/Closed). Render the list using a functional component.

Step 1: Install Node.js and npm

Before you start, ensure you have Node.js and npm (Node Package Manager) installed on your machine.

Download and install the latest version of Node.js.
After installing, you can check the versions of Node.js and npm in the terminal:
node -v
npm -v

Step 2: Create a folder
Create a folder in any location.
Open folder in vscode.

Step 3: Open Integrated vscode Terminal

Type below command in your terminal to create react app.
npx create-react-app issue-tracker
Once the process is complete, you'll see a folder named issue-tracker in your directory. Navigate into the folder:
cd issue-tracker

Step 4: Modify the App.js File and App.css file

Modify the app by editing src/App.js copy the below code and paste it, save it.
Modify the app by editing src/App.css copy the below code and paste it, save it.

App.js:

```
import React from 'react';
import './App.css';
const issues = [
  {
    id: 1,
    title: "Bug in login page",
    description: "The login page throws an error when submitting invalid credentials.",
    status: "Open",
  },
  {
    id: 2,
    title: "UI glitch on homepage",
    description: "There is a UI misalignment issue on the homepage for smaller screens.",
    status: "Closed",
  },
  {
    id: 3,
    title: "Missing translation for settings page",
    description: "The settings page is missing translations for the Spanish language.",
    status: "Open",
  },
  {
    id: 4,
    title: "Database connection error",
    description: "Intermittent database connection issue during peak hours.",
    status: "Open",
  },
];
const Issue = ({ title, description, status }) => {
  return (
    <div className="issue">
      <h3>{title}</h3>
      <p>{description}</p>
      <span className={`status ${status.toLowerCase()}`}>{status}</span>
    </div>
  );
};
```

```
const App = () => {
  return (
    <div className="App">
      <h1>Issue Tracker</h1>
      <div className="issue-list">
        {issues.map((issue) => (
          <Issue
            key={issue.id}
            title={issue.title}
            description={issue.description}
            status={issue.status}
          />
        ))}
      </div>
    </div>
  );
};

export default App;
```

App.css:

```
.App {
  font-family: Arial, sans-serif;
  text-align: center;
  margin: 20px;
}
h1 {
  color: #333;
}
.issue-list {
  display: flex;
  flex-direction: column;
  align-items: center;
}
.issue {
  background-color: #f9f9f9;
  border: 1px solid #ccc;
  border-radius: 5px;
  padding: 15px;
  margin: 10px;
  width: 80%;
  max-width: 600px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}
```

```
.issue h3 {
  margin: 0 0 10px;
  font-size: 1.5em;
}
.issue p {
  margin: 0 0 10px;
  color: #555;
}
.status {
  font-weight: bold;
}
.status.open {
  color: #e74c3c;
}
.status.closed {
  color: #2ecc71;
}
```

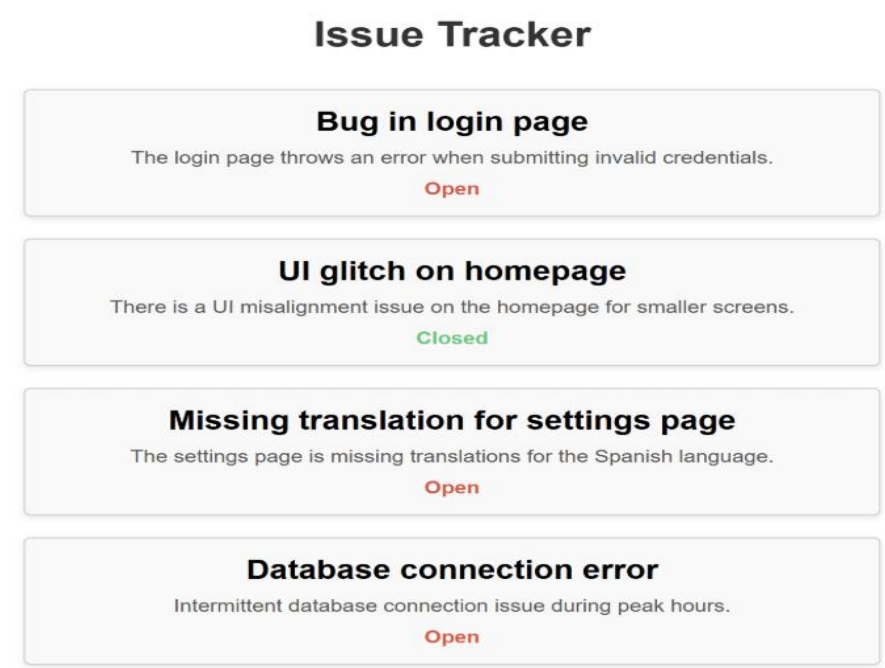
Step 5: Start the Development Server

Now, you can start the development server by using below command

npm start

This will start a local development server and automatically open your new React application in your default web browser. The server will reload the page automatically whenever you make changes to your code.

OUTPUT:



PROGRAM – 7

Install Express (npm install express). Set up a basic server that responds with "Hello, Express!" at the root endpoint (GET /). Create a REST API. Implement endpoints for a Product resource: GET : Returns a list of products. POST : Adds a new product. GET /:id: Returns details of a specific product. PUT /:id: Updates an existing product. DELETE /:id: Deletes a product. Add middleware to log requests to the console. Use express.json() to parse incoming JSON payloads.

Step 1: Create Project

Create a folder and open with vscode.
Open integrated vscode terminal.

Step 2: Install Express

Execute one by one below command.

mkdir express-api
cd express-api
npm init -y
npm install express

Step 3: Create server.js

Create server.js file inside the express-api folder. Right click on the express-api folder and create file server.js.
After then copy the below code and paste it in that server.js file, save it.

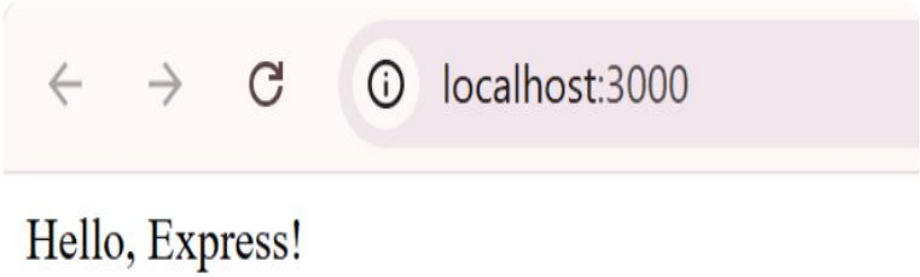
```
const express = require('express');
const app = express();

// Middleware to log requests to the console
app.use((req, res, next) => {
   console.log(`${req.method} request made to: ${req.url}`);
   next();
});

// Middleware to parse JSON payloads
app.use(express.json());

// Root endpoint to return "Hello, Express!"
app.get('/', (req, res) => {
   res.send('Hello, Express!');
});

// Sample in-memory data for products
let products = [
   { id: 1, name: 'Laptop', price: 1000 },
   { id: 2, name: 'Phone', price: 500 }
];
```

```
// GET /products: Returns a list of all products
app.get('/products', (req, res) => {
    res.json(products);
});

// POST /products: Adds a new product
app.post('/products', (req, res) => {
    const { name, price } = req.body;
    if (!name || !price) {
        return res.status(400).json({ message: 'Name and price are required' });
    }
    const newProduct = { id: products.length + 1, name, price };
    products.push(newProduct);
    res.status(201).json(newProduct);
});

// GET /products/:id: Returns details of a specific product
app.get('/products/:id', (req, res) => {
    const product = products.find(p => p.id === parseInt(req.params.id));
    if (!product) {
        return res.status(404).json({ message: 'Product not found' });
    }
    res.json(product);
});

// PUT /products/:id: Updates an existing product
app.put('/products/:id', (req, res) => {
    const product = products.find(p => p.id === parseInt(req.params.id));
    if (!product) {
        return res.status(404).json({ message: 'Product not found' });
    }
    const { name, price } = req.body;
    if (!name || !price) {
        return res.status(400).json({ message: 'Name and price are required' });
    }
    product.name = name;
    product.price = price;
    res.json(product);
});
```

```
// DELETE /products/:id: Deletes a product
app.delete('/products/:id', (req, res) => {
    const productIndex = products.findIndex(p => p.id === parseInt(req.params.id));
    if (productIndex === -1) {
        return res.status(404).json({ message: 'Product not found' });
    }
    products.splice(productIndex, 1);
    res.status(204).send();
});

// Start the server on port 3000
const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Step 5: Run the Server:
node server.js.

OUTPUT:



PROGRAM – 8

Install the MongoDB driver for Node.js. Create a Node.js script to connect to the shop database. Implement insert, find, update, and delete operations using the Node.js MongoDB driver. Define a product schema using Mongoose. Insert data into the products collection using Mongoose. Create an Express API with a /products endpoint to fetch all products. Use fetch in React to call the /products endpoint and display the list of products. Add a POST /products endpoint in Express to insert a new product. Update the Product List, After adding a product, update the list of products displayed in React.

Step 1: Create Project
Create a folder and open with vscode.
Open integrated vscode terminal.

Step 2: Set up the Backend
Run the following commands to set up the environment for your project:
Run one by one command to install the mongoose.
mkdir shop-backend
cd shop-backend

Step 3: Initialize Node.js Project
npm init -y

Step 4: Install Dependencies
npm install express mongoose mongodb cors

Step 5: Create server.js file
Right click on shop-backend folder and create the server.js file.

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose.connect('mongodb://localhost:27017/shop', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const db = mongoose.connection;
db.on('error', console.error.bind(console, 'connection error:'));
db.once('open', () => console.log('Connected to MongoDB'));
```

```
// Define Mongoose Schema for Products
const productSchema = new mongoose.Schema({
  name: String,
  price: Number,
  description: String
});

const Product = mongoose.model('Product', productSchema);

// CRUD API Endpoints

// Fetch all products
app.get('/products', async (req, res) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (error) {
    res.status(500).send(error.message);
  }
});

// Add a new product
app.post('/products', async (req, res) => {
  try {
    const newProduct = new Product(req.body);
    await newProduct.save();
    res.status(201).json(newProduct);
  } catch (error) {
    res.status(400).send(error.message);
  }
});

// Update a product
app.put('/products/:id', async (req, res) => {
  try {
    const updatedProduct = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updatedProduct);
  } catch (error) {
    res.status(400).send(error.message);
  }
});
```

```
// Delete a product
app.delete('/products/:id', async (req, res) => {
  try {
    await Product.findByIdAndDelete(req.params.id);
    res.status(204).send();
  } catch (error) {
    res.status(500).send(error.message);
  }
});

// Start Express server
const PORT = 4000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Step 6: Start the backend

Start the backend server using below command to see in the terminal message server is running on port 4000, connected to MongoDB.

node server.js

Step 6: Set up the Frontend

Open new terminal without deleting running backend server terminal.
Execute 1st command to create frontend folder.
After successfully creating product-client folder then change the directory using 2nd command.

npx create-react-app product-client
cd product-client

Step 7: Edit App.js and App.css file

App.js:

```
import React, { useEffect, useState } from 'react';
import './App.css';

const App = () => {
  const [products, setProducts] = useState([]);
  const [newProduct, setNewProduct] = useState({
    name: '',
    price: '',
    description: ''
  });
  const [editProduct, setEditProduct] = useState(null);
  const [errors, setErrors] = useState({});
```

```
const fetchProducts = () => {
  fetch('http://localhost:4000/products')
    .then(response => response.json())
    .then(data => setProducts(data));
};
useEffect(() => {
  fetchProducts();
}, []);

const handleInputChange = (e) => {
  const { name, value } = e.target;
  if (editProduct) {
    setEditProduct({ ...editProduct, [name]: value });
  } else {
    setNewProduct({ ...newProduct, [name]: value });
  }
  setErrors({ ...errors, [name]: '' });
};

const validateInputs = (product) => {
  const newErrors = {};
  if (!product.name) newErrors.name = 'Product name is required.';
  if (!product.price) newErrors.price = 'Product price is required.';
  if (!product.description) newErrors.description = 'Product description is required.';
  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};

const addProduct = () => {
  if (!validateInputs(newProduct)) return;

  fetch('http://localhost:4000/products', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(newProduct)
  })
    .then(response => response.json())
    .then(() => {
      fetchProducts();
      setNewProduct({ name: '', price: '', description: '' });
    });
};
```

```
const updateProduct = () => {
  if (!validateInputs(editProduct)) return;

  fetch(`http://localhost:4000/products/${editProduct._id}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(editProduct)
  })
    .then(response => response.json())
    .then(() => {
      fetchProducts();
      setEditProduct(null);
    });
};

const deleteProduct = (productId) => {
  if (window.confirm('Are you sure you want to delete this product?')) {
    fetch(`http://localhost:4000/products/${productId}`, {
      method: 'DELETE'
    })
      .then(() => fetchProducts());
  }
};

return (
  <div className="container">
    <h1 className="title">Product Management</h1>

    {products.length === 0 ? (
      <p className="no-products-message">No products available. Please add a new product.</p>
    ) : (
      <table className="product-table">
        <thead>
          <tr>
            <th>Name</th>
            <th>Price</th>
            <th>Description</th>
            <th>Actions</th>
          </tr>
        </thead>
```

```
        <tbody>
          {products.map(product => (
            <tr key={product._id} className="product-row">
              <td>{product.name}</td>
              <td>₹{product.price}</td>
              <td>{product.description}</td>
              <td>
                <div className="action-buttons">
                  <button onClick={() => setEditProduct(product)} className="edit-button">Edit</button>
                  <button onClick={() => deleteProduct(product._id)} className="delete
button">Delete</button>
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    )}

    <div className="form-section">
      <h2 className="form-title">{editProduct ? 'Edit Product' : 'Add New Product'}</h2>

      <div className="input-group">
        <div className="input-wrapper">
          <input
            type="text"
            name="name"
            value={editProduct ? editProduct.name : newProduct.name}
            onChange={handleInputChange}
            placeholder="Product Name"
            className="input-field"
          />
          {errors.name && <p className="error-message">{errors.name}</p>}
        </div>

        <div className="input-wrapper">
          <input
            type="number"
            name="price"
            value={editProduct ? editProduct.price : newProduct.price}
            onChange={handleInputChange}
            placeholder="Product Price ₹"
            className="input-field"
          />
```

```
          {errors.price && <p className="error-message">{errors.price}</p>}
        </div>

        <div className="input-wrapper">
          <textarea
            name="description"
            value={editProduct ? editProduct.description : newProduct.description}
            onChange={handleInputChange}
            placeholder="Product Description"
            className="input-field"
            rows={3}
          />
          {errors.description && <p className="error-message">{errors.description}</p>}
        </div>
      </div>

      {editProduct ? (
        <button onClick={updateProduct} className="update-button">Update Product</button>
      ) : (
        <button onClick={addProduct} className="add-button">Add Product</button>
      )}
    </div>
  </div>
  );
};

export default App;
```

App.css:

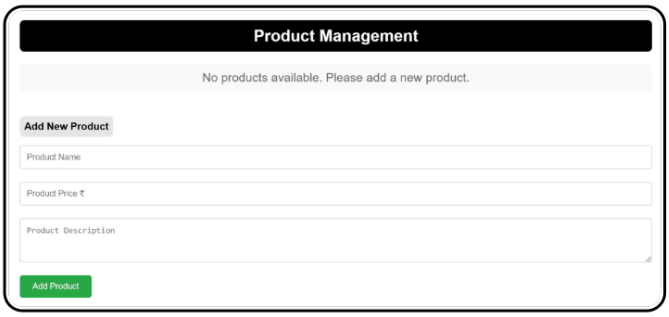
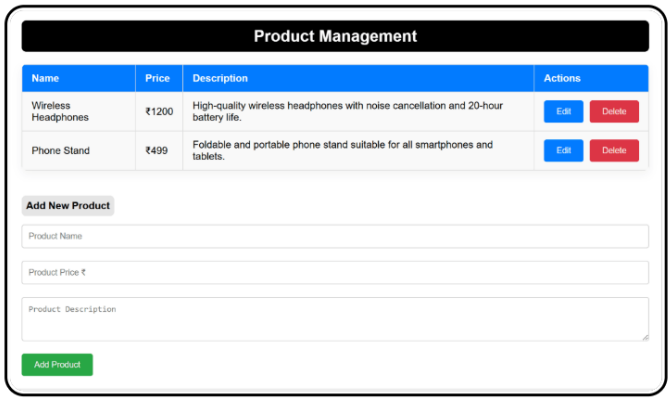
```
.container {
  max-width: 1000px;
  margin: 0 auto;
  padding: 20px;
  font-family: Arial, sans-serif;
  background-color: #ffffff;
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
  border-radius: 8px;
}
```

```
.title {
  border-radius: 7px;
  color: #fff;
  background: #000;
  padding: 10px 0;
  font-size: 1.6rem;
  text-align: center;
  margin: 0;
}

.product-table {
  width: 100%;
  border-collapse: collapse;
  margin: 20px 0;
  background-color: #f9f9f9;
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.05);
  border-radius: 8px;
  overflow: hidden;
}

.product-table thead {
  background-color: #007bff;
  color: white;
  text-align: left;
  font-size: 1rem;
}

.product-table th,
.product-table td {
  padding: 12px 15px;
  border: 1px solid #ddd;
}

.product-table tr {
  transition: background-color 0.2s ease-in-out;
}

td button {
  display: block;
}

.product-table tbody tr:nth-child(even) {
  background-color: #f9f9f9;
}
```

```
.input-group {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.input-field {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.add-button,
.edit-button,
.update-button {
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.add-button {
  font-weight: 500;
  margin-top: 10px;
  background-color: #28a745;
  color: white;
}

.add-button:hover {
  background-color: #218838;
}

.edit-button {
  background-color: #007bff;
  color: white;
}

.edit-button:hover {
  background-color: #0056b3;
}
```

```
.update-button {
  margin-top: 20px;
  background-color: #ffc107;
  color: black;
  font-weight: 600;
}

.update-button:hover {
  background-color: #e0a800;
}

.action-buttons {
  display: flex;
  gap: 10px;
  justify-content: center;
  align-items: center;
}

.form-section {
  margin-top: 40px;
}

.form-title {
  width: fit-content;
  background: #0000001a;
  padding: 7px 7px;
  border-radius: 7px;
  font-size: 16px;
  color: #000000;
}

.delete-button {
  background-color: #dc3545;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.delete-button:hover {
  background-color: #c82333;
}
```

```
.no-products-message {
  text-align: center;
  font-size: 1.2rem;
  color: #666;
  margin-top: 20px;
  padding: 10px;
  background-color: #f9f9f9;
  border-radius: 8px;
}

.error-message {
  color: red;
  background-color: #ffe6e6;
  padding: 5px;
  border-radius: 5px;
  margin-top: 5px;
  font-size: 0.9rem;
}

.input-wrapper {
  display: flex;
  flex-direction: column;
  justify-content: center;
  margin-bottom: 10px;
}
```

Step 8: Start the development server
npm start

OUTPUT: